SISTEMAS DISTRIBUÍDOS

Processos

Prof Priscila Marques Kai

- O conceito de um **processo** é originário do campo de sistemas operacionais
  - Processo é um programa em execução
    - Um processo possui espaço próprio de endereço, consistindo em:
      - Região de texto
      - Região de dados
      - Região de pilha

Introdução

- Necessidade de gerenciamento e escalonamento de processos
  - Processo passa por uma série de estados de processo:
    - Estado de execução
    - Estado de pronto
    - Estado de bloqueado
- Em Sistemas Distribuídos usar técnicas de multithreading aumentam o desempenho

Processador virtual

- Para cada programa em execução, o sistema operacional cria um processador virtual no topo dos processadores físicos.
  - O SO materializa o processo através do **bloco de controle de processo – PCB**
    - PCBs mantêm as informações que o sistema operacional precisa para gerenciar o processo

Bloco de controle de processo – PCB

- O PCB de todos os processos ativos residem na memória principal em uma área exclusiva do SO
- O SO mantém ponteiros para cada PCB do processo em uma tabela de processos

ponteiros
Estado do processo
Nome do processo
Prioridade do processo
Registradores
Limites de memória
Lista de arquivos abertos
. . .

Gerência do processo

- A gerência de processos é feita por meio de **chamadas ao sistema operacional** (rotinas do SO), que cuidam de tarefas como **criar**, **encerrar** e **sincronizar** processos.
  - Ao mesmo tempo, **vários processos** usam a **CPU** e outros recursos de hardware **ao mesmo tempo**, e o SO faz esse compartilhamento de forma **transparente**, como se cada processo tivesse acesso exclusivo aos recursos.

Processos e Threads

- Processo: um programa em execução, com seu próprio espaço de memória (endereçamento) e recursos.
- Thread: uma linha de execução dentro de um processo (um “fluxo” que roda tarefas).

Escalonamento

- O escalonador decide qual processo/thread usa a CPU a cada instante.
  - Escalonamento: Preemptivo e Não-Preemptivo.

Threads em Sistemas não distribuídos

- Multiprogramação: vários processos simultâneos disputando o processador

- Intermediação da disputa: Algoritmos de Escalonamento

- Necessidade de Representar um processo:
  - Implementação de Processos → Estruturas de Dados (Bloco de Controle de Processos - BCP)

Threads em Sistemas não distribuídos

- Um processo pode executar partes do código concorrentemente ou em paralelo.
  - Múltiplas threads tornam aplicações concorrentes mais eficientes.

Vantagens do uso de threads

- Criação e finalização mais rápidas do que processos.
- Compartilhamento de dados mais eficiente: mesma memória (mesmo espaço de endereçamento).

Desafios e cuidados

- Não há proteção automática entre threads: elas compartilham memória.
- Um thread pode afetar dados de outro (se o código não for cuidadoso).
- Proteger dados (evitar condições de corrida) fica a cargo do desenvolvedor.

Threads em Sistemas não distribuídos

- Threads permitem dividir o trabalho em tarefas concorrentes, evitando que todo o processo fique bloqueado esperando uma resposta (ex.: E/S, rede, disco).
- Com CPUs multicore mais acessíveis, tornou-se comum usar multithreading para servidores e aplicações que precisam atender muitos pedidos ao mesmo tempo.
- Em várias situações, o ganho vem de:
  - manter a aplicação responsiva,
  - aproveitar melhor a CPU enquanto uma parte espera E/S.

Threads em Sistemas não distribuídos

Em aplicações grandes, é comum dividir o sistema em programas cooperativos rodando em processos diferentes.

- A cooperação entre processos ocorre via IPC (InterProcess Communication), como:
  - pipes, filas, sockets, memória compartilhada, sinais, etc.

- A principal desvantagem de IPC é que a comunicação muitas vezes requer extensivo chaveamento de contexto:
  - Quando dois processos precisam se comunicar, o sistema operacional precisa intervir (porque processos comuns não acessam diretamente a memória uns dos outros, por questões de proteção e isolamento).
  - Gera várias trocas de contexto.

Threads em Sistemas não distribuídos

Exemplo para ilustrar diferenças entre Processos e Threads
Exemplo 1 com dois processos

```
from multiprocessing import Process
from time import *
from random import *
```

Process: classe Process, usada para criar e controlar **novos processos.**

Threads em Sistemas não distribuídos

Exemplo para ilustrar diferenças entre Processos e Threads
Exemplo 1 com dois processos

```
def sleeper(name): # função que representa a tarefa a ser executada
    s=randint(1,20)
    t=localtime()
    txt=str(t.tm_hour)+':'+str(t.tm_min)+':'+str(t.tm_sec)+' '+ name +'
is going to sleep for '+str(s)+' seconds'
    print(txt)
    sleep(s)
    t=localtime()
    txt=str(t.tm_hour)+':'+str(t.tm_min)+':'+str(t.tm_sec)+' '+ name +'
has woken up'
    print(txt)
```

Threads em Sistemas não distribuídos

Exemplo para ilustrar diferenças entre Processos e Threads
Exemplo 1 com dois processos

```
if __name__ == "__main__":
    p=Process(target=sleeper,args=('eve',))
    q=Process(target=sleeper,args=('bob',))
    p.start();q.start()
    p.join();q.join()
```

Threads em Sistemas não distribuídos

Exemplo para ilustrar diferenças entre Processos e Threads
Exemplo 2 com multithreads (3 no total)

```
from multiprocessing import Process
from random import *
from threading import Thread
from time import *
```

Threads em Sistemas não distribuídos

Exemplo para ilustrar diferenças entre Processos e Threads
Exemplo 2 com multithreads (3 no total)

```
shared_x=randint(10,99)

def sleeping(name):
    global shared_x
    s=randint(1,20)
    t=localtime()
    txt=str(t.tm_min)+':'+str(t.tm_sec)+' '+ name +' is going to sleep for '+str(s)+' seconds'
    print(txt)
    sleep(s)
    t=localtime()
    shared_x=shared_x+1
    txt=str(t.tm_min)+':'+str(t.tm_sec)+' '+name+' has woken up, seeing shared_x being + str(shared_x)
    print(txt)
```

Threads em Sistemas não distribuídos

Exemplo para ilustrar diferenças entre Processos e Threads
Exemplo 2 com multithreads (3 no total)

```
def sleeper(name):
    sleeplist=list()
    for i in range(3):
        subsleeper=Thread(target=sleeping,args=(name+' '+str(i),))
        sleeplist.append(subsleeper)
    for s in sleeplist:s.start()
    for s in sleeplist:s.join()
```

Threads em Sistemas não distribuídos

Exemplo para ilustrar diferenças entre Processos e Threads
Exemplo 2 com multithreads (3 no total)

```
if __name__ == "__main__":
    p=Process(target=sleeper,args=('eve',))
    q=Process(target=sleeper,args=('bob',))
    print('eve', 'seeing shared_x being', shared_x)
    print('bob', 'seeing shared_x being', shared_x)
    p.start();q.start()
    p.join();q.join()
```

Qual o resultado observado?

Threads - Sincronização entre processos

Para compartilhar shared_x entre processos, precisamos de algo que suporte memória compartilhada, como por exemplo o Value:

`multiprocessing.Value`

Threads - Sincronização entre processos

```
def sleeping(name, shared_x):
    s=randint(1,20)
    t=localtime()
    txt=str(t.tm_min)+':'+str(t.tm_sec)+' '+ name +' is going to sleep for
'+str(s)+' seconds'
    print(txt)
    sleep(s)
    t=localtime()
    with shared_x.get_lock():
        shared_x.value += 1
    txt=str(t.tm_min)+':'+str(t.tm_sec)+' '+name+' has woken up, seeing
shared_x being '+ str(shared_x)
    print(txt)
```

Threads - Sincronização entre processos

Para compartilhar shared_x entre processos, precisamos de algo que suporte memória compartilhada, como por exemplo o Value:

`multiprocessing.Value`

- **`get_lock()`**: Usado com objetos compartilhados (`multiprocessing`) ou bibliotecas que já possuem seu próprio mecanismo de bloqueio interno e você quer reutilizá-lo.

Threads - Sincronização entre processos

Para compartilhar shared_x entre processos, precisamos de algo que suporte memória compartilhada, como por exemplo o Value:

`multiprocessing.Value`

- **`get_lock()`**: Usado com objetos compartilhados (`multiprocessing`) ou bibliotecas que já possuem seu próprio mecanismo de bloqueio interno e você quer reutilizá-lo.

Threads x Processos - Concorrência

Threads

- São leves e rápidas.
- Compartilham o mesmo espaço de memória → isso facilita a comunicação, mas torna a programação mais difícil, porque o programador precisa gerenciar a sincronização.
- Um erro em uma *thread* pode afetar todas as outras (por exemplo, sobrescrever dados compartilhados).

Processos

- São mais pesados que threads, mas têm memória isolada.
- Cada processo trabalha no seu espaço de dados, evitando interferência.
- O sistema operacional, com ajuda do hardware, impede que um processo leia ou altere a memória de outro.
- Comunicação entre processos é mais trabalhosa (precisa de IPC – *Interprocess Communication*), mas mais segura.

Threads em Sistemas Distribuídos

- Uma das vantagens do uso de *threads* é:
  - Proporcionar um meio conveniente para permitir chamadas bloqueadoras de sistema sem bloquear o processo inteiro no qual o thread está executando
- Esta propriedade torna os *threads* atraentes para o uso em sistemas distribuídos
  - Facilitam a comunicação, podendo manter múltiplas conexões lógicas ao mesmo tempo

Clientes MultiThreads

Em sistemas distribuídos que funcionam em redes de longa distância (ex.: Internet), a latência (demora para enviar e receber mensagens) pode ser muito alta – às vezes centenas de milissegundos ou até segundos.

Como resolver?

Uma técnica comum é não esperar parado pela resposta:

- enquanto uma requisição é feita, o programa continua executando outras tarefas.

Exemplo: Navegadores Web

- Uma página da Web normalmente é formada por um arquivo HTML + várias imagens, ícones, scripts etc.
- Para cada elemento, o navegador precisa abrir uma conexão com o servidor, baixar os dados e exibir.
- Se fosse sequencial, o usuário teria que esperar tudo ser baixado antes de ver a página.

Com threads:

1. O navegador baixa o HTML principal.
2. Assim que o HTML chega, o navegador já começa a mostrar o texto.
3. Threads separadas cuidam de buscar as imagens, ícones etc.
4. Enquanto isso, o usuário já pode rolar a página e interagir.

Resultado: o carregamento parece mais rápido e fluido, mesmo que as imagens demorem a chegar.

Servidores MultiThreads

O *multithreading* em servidores possui muitos benefícios.

- Facilita a programação e aumenta o desempenho, aproveitando o paralelismo — especialmente em máquinas multicore, como também em processadores mais simples.

Exemplo: Servidor de Arquivos

- O servidor recebe requisições de clientes para ler ou escrever arquivos no disco. Problema: acessar disco é lento → pode deixar a CPU parada enquanto espera.

Exemplo: Servidor de Arquivos

- O servidor recebe requisições de clientes para ler ou escrever arquivos no disco. Problema: acessar disco é lento → pode deixar a CPU parada enquanto espera.

Servidor *Single-threaded* (sequencial)

- O servidor tem apenas uma thread.
  - Fluxo: recebe uma requisição → executa completamente → só então atende a próxima.
  - Problema:
    - Enquanto espera o disco (bloqueio), ele não atende outros clientes.
    - CPU pode ficar ociosa durante essa espera.
  - Consequência: baixa taxa de requisições atendidas por segundo.

Exemplo: Servidor de Arquivos

- O servidor recebe requisições de clientes para ler ou escrever arquivos no disco. Problema: acessar disco é lento → pode deixar a CPU parada enquanto espera.

Servidor *Multithreaded*

- Thread despachante: recebe as requisições dos clientes.
- Threads operárias: cada uma cuida de um pedido específico.
- Funcionamento:
  - O despachante entrega uma requisição para um operário.
  - Se o operário precisar ler do disco (operação bloqueante), apenas aquela thread pausa.
  - As outras threads continuam trabalhando → a CPU não fica parada.

- Modelo despachante/operário

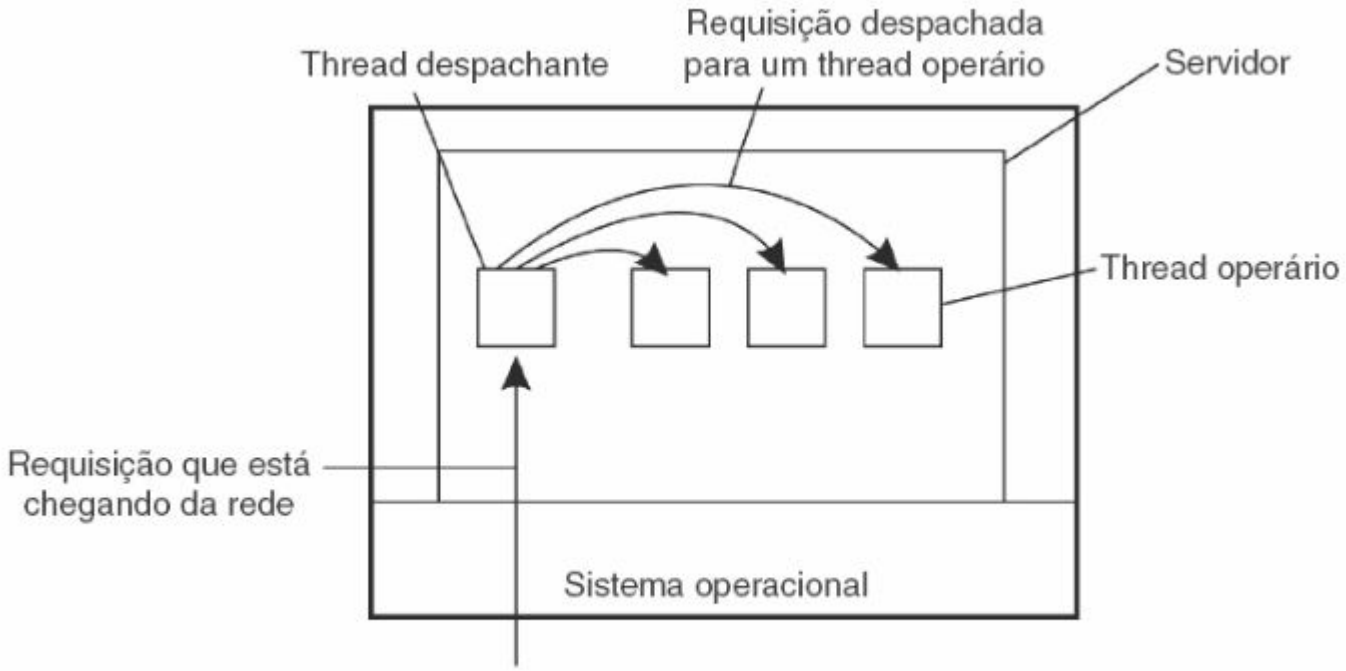

Figura 3.3 Servidor multithread organizado segundo modelo despachante/operário.

Servidor *Single-threaded* com Máquina de Estados (*event-driven*)

Outra forma é usar apenas uma *thread*, mas sem bloquear.

- Quando precisa do disco, faz uma chamada não bloqueante (assíncrona).
- Registra o estado atual da requisição e continua atendendo outros pedidos.
- Quando o SO avisa que o disco terminou a operação, o servidor retoma a requisição de onde parou.

Vantagens:

- Bom para sistemas com muitas conexões simultâneas (ex.: servidores de chat, jogos online).

Desvantagem:

- Programação mais complexa: o desenvolvedor precisa gerenciar manualmente todos os estados e eventos → mais difícil de escrever e manter.

Servidores MultiThreads

Como podemos aplicar a idéia de multitheading em um servidor?

Exemplo: SD Vestuário: API de Loja

- Aplicação web com Flask que expõe uma API HTTP
- Simula uma loja com:
  - produtos (estoque)
  - pedidos (histórico)
- Banco de dados em memória, contendo:
  - produtos: nome → {código, nome, quantidade}
  - pedidos: lista de pedidos realizados
  - proximo_pedido_id: gera IDs sequenciais

```
banco = {
    "produtos": {
        "camisa": {"codigo": "CP000001",
        "nome": "camisa",
        "quantidade": 10},
        "calca": {"codigo": "CP000002",
        "nome": "calca",
        "quantidade": 8},
        "vestido": {"codigo":
"CP000003",
        "nome": "vestido",
        "quantidade": 5}
    },
    "pedidos": [],
    "proximo_pedido_id" : 1
}
```

Servidores MultiThreads

Como podemos aplicar a idéia de multitheading em um servidor?

Exemplo: SD Vestuário: API de Loja

threaded=True

- O servidor pode atender mais de um cliente ao mesmo tempo
- Cada requisição pode ser tratada por uma thread
- Isso melhora:
  - capacidade de atendimento
  - tempo de resposta em cenários concorrentes

```
if __name__ == "__main__":
    app.run(threaded=True)
```

Servidores MultiThreads

Como podemos aplicar a idéia de multitheading em um servidor?

Exemplo: SD Vestuário: API de Loja

Lidando com concorrência no banco em memória

- Várias requisições podem acessar/modificar o dicionário banco ao mesmo tempo

  Exemplo:

  - dois clientes compram o mesmo produto “ao mesmo tempo”
  - sem controle, pode ocorrer estoque negativo ou pedidos inconsistentes

  Como tratar?

  - Criar manualmente um mecanismo de bloqueio para proteger a seção crítica do código

Como podemos aplicar a idéia de multitheading em um servidor?

Exemplo: SD Vestuário: API de Loja

Como tratar?

- Criar manualmente um mecanismo de bloqueio para proteger a seção crítica do código

  Uso de `Lock()` cria um mecanismo de exclusão mútua

  `with lock_banco`: garante que:

  - apenas uma thread por vez acessa/modifica o banco evitando condições de corrida (race conditions)

```
lock_banco = Lock()
...
with lock_banco:
        total_produtos = len(banco["produtos"])
        total_pedidos = len(banco["pedidos"])
```

Servidores MultiThreads

Como podemos aplicar a idéia de multitheading em um servidor?

Exemplo: SD Vestuário: API de Loja

`with lock_banco`: garante que:

- apenas uma thread por vez acessa/modifica o banco evitando condições de corrida (race conditions)
- Em rotas GET possibilita a leitura protegida
    - Mesmo leitura deve ser protegida quando há escrita concorrente
    - evita ler enquanto acontece uma atualização